

IPv4 Addressing and Subnet Design

Laboratory 1

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	4 to 5 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name Agbo Daniel Ameh

Registration number c1126OCO17757

Submission date 09/30/2026

Purpose and Scenario

This laboratory develops the addressing skills required to configure, verify and troubleshoot IPv4 networks. You will connect binary addressing to CIDR, calculate usable ranges, build a variable length subnet plan and verify the result on Linux.

You are preparing an address plan for a small training network. The network team has assigned 192.168.50.0/24 and requires subnets for groups of different sizes. Before deployment, you must prove every network ID, usable range and broadcast address and safely test representative addresses on a temporary Linux interface.

Learning Outcomes

- Identify IPv4 interface addresses, routes and the default gateway on Linux.
- Explain the historical purpose of Class A, B and C addressing without using classful rules for modern subnet design.
- Convert IPv4 octets between decimal and binary.
- Calculate subnet masks, block sizes, network IDs, broadcasts and usable ranges from CIDR prefixes.
- Create equal-size and variable length subnets that meet stated host requirements.
- Configure and remove IPv4 addresses on a temporary Linux dummy interface.
- Verify subnet calculations with Python's standard ipaddress module.

Environment and Prerequisites

- A Kali Linux or Ubuntu Linux virtual machine with console access.
- A snapshot taken before the lab.
- A user account with sudo permission.
- Python 3 and a text editor.
- The CORE102 pre-lab lecture materials.

Safety and Authorisation

- Use the temporary dummy interface created in this manual. Do not change the address of your active internet interface.
- Read each interface name before using sudo. A wrong interface may disconnect the virtual machine.
- Do not assign a laboratory address to a production or institutional network.
- Remove the dummy interface during cleanup and record the cleanup evidence.

Evidence Requirements

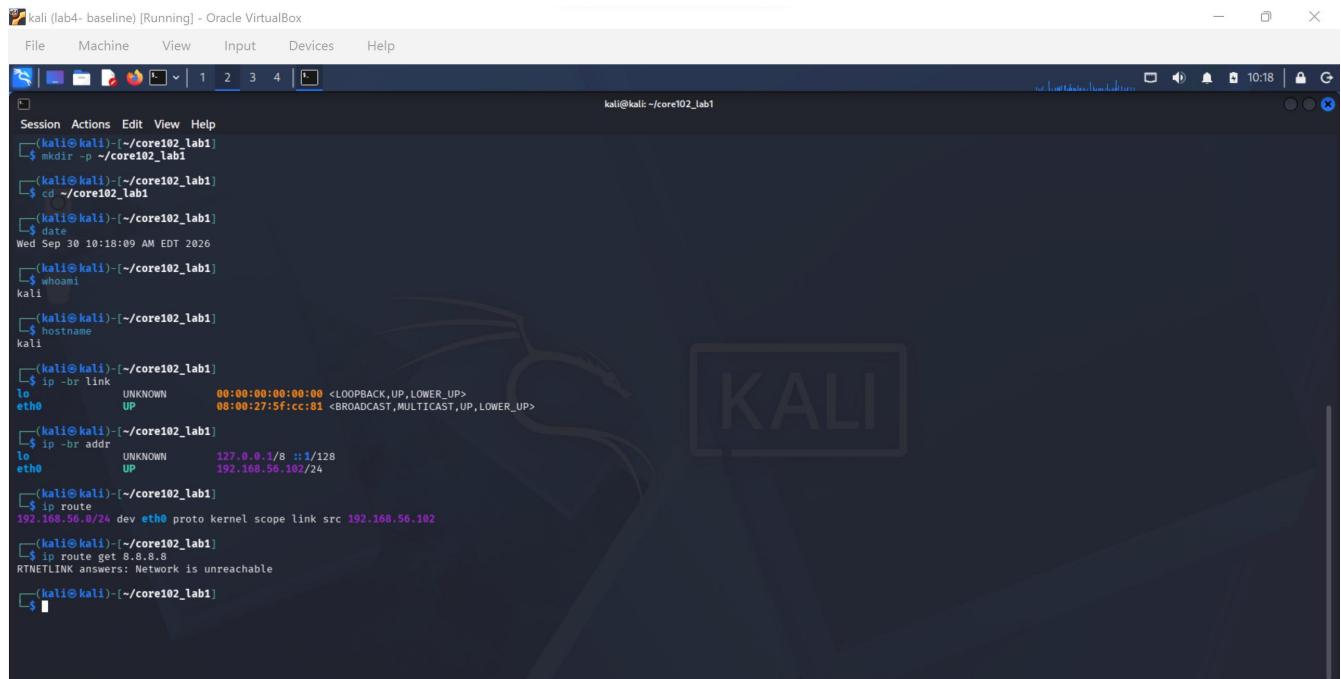
Item	Required evidence	Included
1	Baseline terminal record showing date, identity, host name, addresses and routes.	[]

Item	Required evidence	Included
2	Completed binary and CIDR calculation tables.	☐
3	Screenshots showing the dummy interface configuration and cleanup.	☐
4	Completed equal-size subnet and VLSM designs.	☐
5	Working subnet_report.py script and its output.	☐
6	Short explanations and reflection responses in the PDF report.	☐

Part 1 Network Baseline

Observation	Your result	Meaning
Active interface	eth0	Interface used by the default route
IPv4 address and prefix	192.168.51.102/24	Host address and network prefix
Default gateway	Not configure	Router used for destinations outside the local subnet
Route source address	Not returned ip route get 8.8.8.8 gave Network not reachable	Address Linux selected for the test destination

EVIDENCE



```
kali (lab4- baseline) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~/core102_lab1
Session Actions Edit View Help
$ mkdir -p ~/core102_lab1
$ cd ~/core102_lab1
$ date
Wed Sep 30 10:18:09 AM EDT 2026
$ whoami
kali
$ hostname
kali
$ ip -br link
to UNKNOWN 00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0 UP 08:00:27:5f:cc:81 <BROADCAST,MULTICAST,UP,LOWER_UP>
$ ip -br addr
to UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.56.102/24
$ ip route
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.102
$ ip route get 8.8.8.8
RTNETLINK answers: Network is unreachable
$
```

Report

I created the working directory ~/core102_lab1 and captured the baseline network state before making any changes. The active interface is eth0 with IPv4 address 192.168.56.102/24, on the private lab subnet. ip route

shows only a directly-connected route for 192.168.56.0/24 there is no default gateway configured on this VM, which is consistent with the host-only adapter used in the lab environment. Because no default route exists, ip route get 8.8.8.8 returned RTNETLINK answers: Network is unreachable rather than selecting a source address: Linux had no path to external destinations to evaluate. This baseline confirms the active interface and its prefix, and clarifies that any external connectivity test would fail by design in this topology.

Part 2 Binary and Historical Address Classes

Classful addressing is included because it explains older documentation and default masks. Modern networks use CIDR prefixes. Always use the stated prefix when one is provided.

Historical class	First octet	Default prefix	Default mask
A	1 to 126	/8	255.0.0.0
B	128 to 191	/16	255.255.0.0
C	192 to 223	/24	255.255.255.0

Address	Historical class	Given prefix	Network ID	Broadcast
10.24.6.9/20	A	/20	/20	10.24.15.255
172.20.14.200/23	B	/23	/23	172.20.15.255
192.168.44.77/27	C	/27	/27	192.168.44.95
198.51.100.130/26	D	/26	/26	198.51.100.191

Evidence

```
kali (Snapshot 2) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
kali@kali:~$ python3 -c <<PY
for value in (192, 168, 10, 25):
    print(f'{value:3} = {value:08b}')
PY
192 = 11000000
168 = 10101000
10 = 00001010
25 = 00011001
kali@kali:~$ 11000000.10101000.00001010.00011001
11000000.10101000.00001010.00011001: command not found
kali@kali:~$
```

Report

I converted 192, 168, 10 and 25 to 8-bit binary using Python's :08b format specifier, confirming 192.168.10.25 is 11000000.10101000.00001010.00011001. For each address in the table I recorded the historical class and default mask, then re analysed the address using the explicitly stated CIDR prefix. The historical class describes the default mask the address would have had under classful rules — it does not constrain the actual prefix used. For example, 10.24.6.9 is a class A address (default /8), but with the given /20 prefix its network ID is 10.24.0.0 and broadcast is 10.24.15.255, not the whole 10.0.0.0/8 block. Similarly, 172.20.14.200 is historically class B (/16) but deployed as /23, and both 192.168.44.77 and 198.51.100.130 are class C by default (/24) yet deployed as /27 and /26 respectively.

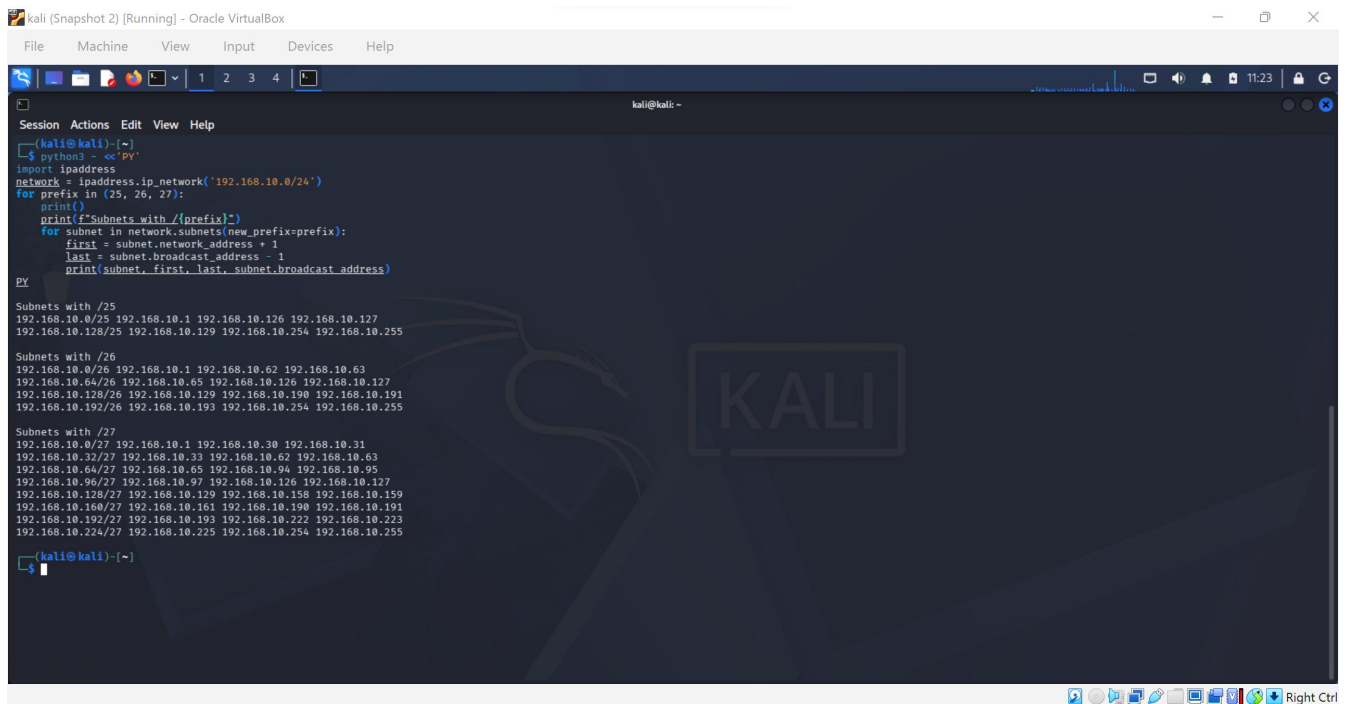
Part 3 CIDR and Equal Size Subnetting

For IPv4, host bits equal 32 minus the prefix length. A subnet contains 2 raised to the number of host bits addresses. Traditional usable-host calculations subtract the network and broadcast addresses.

Prefix	Mask	Block size	Total addresses	Traditional usable hosts
/24	255.255.255.0	256	256	254
/25	255.255.255.128	128	128	126
/26	255.255.255.192	64	64	62
/27	255.255.255.224	32	32	30
/28	255.255.255.240	16	16	14
/29	255.255.255.248	8	8	6
/30	255.255.255.252	4	4	2

Subnet	Network ID	First host	Last host	Broadcast
1				
2				
3				
4				

Evidence



```
kali (Snapshot 2) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
$ python3 -c 'import ipaddress; network = ipaddress.ip_network("192.168.10.0/24"); for prefix in (25, 26, 27): print(f"Subnets with /{prefix}"); for subnet in network.subnets(new_prefix=prefix): first = subnet.network_address + 1; last = subnet.broadcast_address - 1; print(subnet, first, last, subnet.broadcast_address)'
```

Subnets with /25
192.168.10.0/25 192.168.10.1 192.168.10.126 192.168.10.127
192.168.10.128/25 192.168.10.129 192.168.10.254 192.168.10.255

Subnets with /26
192.168.10.0/26 192.168.10.1 192.168.10.62 192.168.10.63
192.168.10.64/26 192.168.10.65 192.168.10.126 192.168.10.127
192.168.10.128/26 192.168.10.129 192.168.10.190 192.168.10.191
192.168.10.192/26 192.168.10.193 192.168.10.254 192.168.10.255

Subnets with /27
192.168.10.0/27 192.168.10.1 192.168.10.30 192.168.10.31
192.168.10.32/27 192.168.10.33 192.168.10.62 192.168.10.63
192.168.10.64/27 192.168.10.65 192.168.10.94 192.168.10.95
192.168.10.96/27 192.168.10.97 192.168.10.126 192.168.10.127
192.168.10.128/27 192.168.10.129 192.168.10.158 192.168.10.159
192.168.10.160/27 192.168.10.161 192.168.10.190 192.168.10.191
192.168.10.192/27 192.168.10.193 192.168.10.222 192.168.10.223
192.168.10.224/27 192.168.10.225 192.168.10.254 192.168.10.255

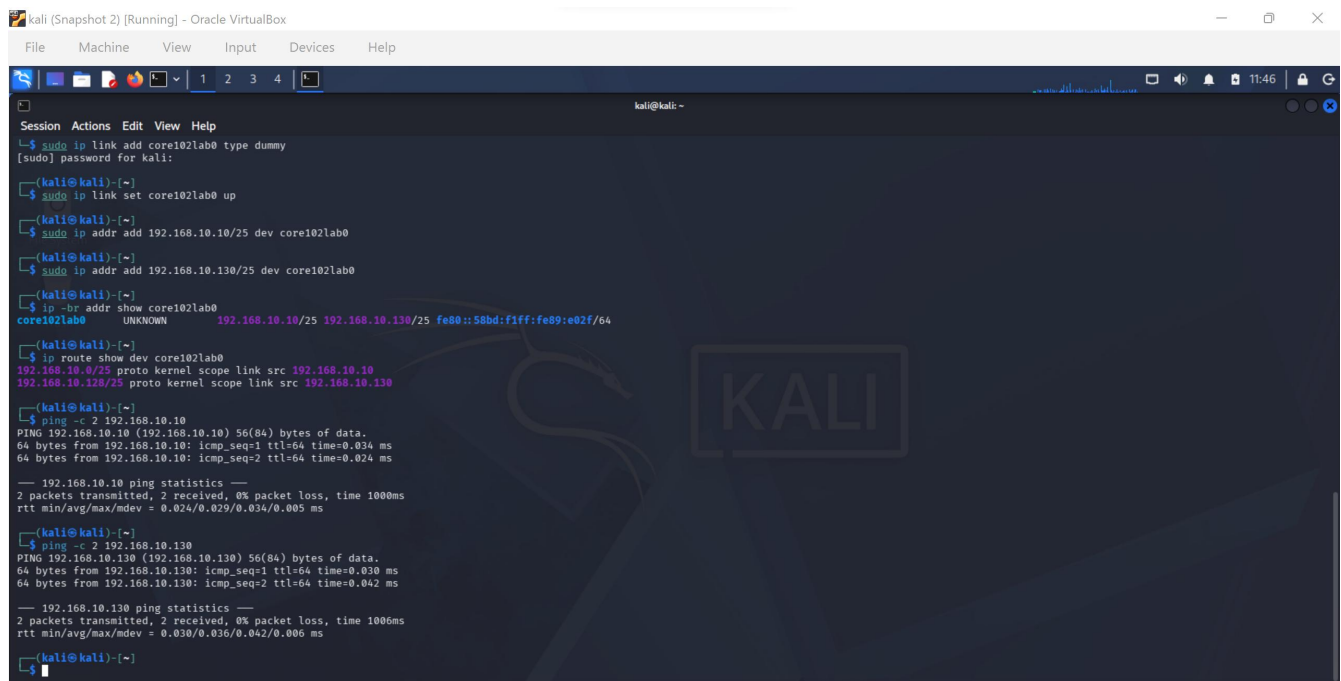
Report

I used Python's ipaddress module to subdivide 192.168.10.0/24 into /25, /26 and /27 subnets and cross-checked each result against manual calculations. For /25 the block size is 128, giving subnets at .0 (usable .1–.126, broadcast .127) and .128 (usable .129–.254, broadcast .255). For /26 the block size is 64, giving four subnets at .0, .64, .128 and .192 with broadcasts .63, .127, .191 and .255. For /27 the block size is 32, giving eight subnets at .0, .32, .64, .96, .128, .160, .192 and .224.

The address 192.168.10.173 falls into the sixth /27 subnet — 192.168.10.160/27 — because .160 is the largest block start below .173 and .191 is its broadcast. All Python output matched my manual work with no mismatches; had they differed, the most likely cause would have been using the wrong block size when stepping between subnets (for example stepping by 32 instead of 64 when dividing into /26), which is the classic cause of subnet overlap.

Part 4 Safe Address Configuration Evidence

```
(kali@kali)-[~]
$ ip link show core102lab0 2>/dev/null || echo "Name is available"
Name is available
(kali@kali)-[~]
$ sudo ip link add core102lab0 type dummy
[sudo] password for kali:
(kali@kali)-[~]
$ sudo ip link set core102lab0 up
(kali@kali)-[~]
$ sudo ip addr add 192.168.10.10/25 dev core102lab0
(kali@kali)-[~]
$ sudo ip addr add 192.168.10.130/25 dev core102lab0
(kali@kali)-[~]
$ ip -br addr show core102lab0
core102lab0      UNKNOWN      192.168.10.10/25 192.168.10.130/25 fe80::58bd:f1ff:fe89:e02f/64
(kali@kali)-[~]
```



```
kali (Snapshot 2) [Running] - Oracle VM VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
$ sudo ip link add core102lab0 type dummy
[sudo] password for kali:
(kali@kali)-[~]
$ sudo ip link set core102lab0 up
(kali@kali)-[~]
$ sudo ip addr add 192.168.10.10/25 dev core102lab0
(kali@kali)-[~]
$ sudo ip addr add 192.168.10.130/25 dev core102lab0
(kali@kali)-[~]
$ ip -br addr show core102lab0
core102lab0      UNKNOWN      192.168.10.10/25 192.168.10.130/25 fe80::58bd:f1ff:fe89:e02f/64
(kali@kali)-[~]
$ ip route show dev core102lab0
192.168.10.0/25 proto kernel scope link src 192.168.10.10
192.168.10.128/25 proto kernel scope link src 192.168.10.130
(kali@kali)-[~]
$ ping -c 2 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data:
64 bytes from 192.168.10.10: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=0.024 ms

--- 192.168.10.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1000ms
rtt min/avg/max/mdev = 0.024/0.029/0.034/0.005 ms
(kali@kali)-[~]
$ ping -c 2 192.168.10.130
PING 192.168.10.130 (192.168.10.130) 56(84) bytes of data:
64 bytes from 192.168.10.130: icmp_seq=1 ttl=64 time=0.030 ms
64 bytes from 192.168.10.130: icmp_seq=2 ttl=64 time=0.042 ms

--- 192.168.10.130 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 0.030/0.036/0.042/0.006 ms
(kali@kali)-[~]
```

```
kali (Snapshot 2) [Running] - Oracle VirtualBox
File Machine View Input Devices Help

kali@kali: ~
Session Actions Edit View Help
~$ sudo ip addr add 192.168.10.10/25 dev core102lab0
(kali@kali)~$
~$ sudo ip addr add 192.168.10.130/25 dev core102lab0
(kali@kali)~$
~$ ip -br addr show core102lab0
core102lab0 UNKNOWN 192.168.10.10/25 192.168.10.130/25 fe80::58bd:f1ff:fe89:e02f/64
(kali@kali)~$
~$ ip route show dev core102lab0
192.168.10.0/25 proto kernel scope link src 192.168.10.10
192.168.10.128/25 proto kernel scope link src 192.168.10.130
(kali@kali)~$
~$ ping -c 2 192.168.10.10
PING 192.168.10.10 (192.168.10.10) 56(84) bytes of data:
64 bytes from 192.168.10.10: icmp_seq=1 ttl=64 time=0.034 ms
64 bytes from 192.168.10.10: icmp_seq=2 ttl=64 time=0.024 ms

--- 192.168.10.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1000ms
rtt min/avg/max/mdev = 0.024/0.029/0.034/0.005 ms
(kali@kali)~$
~$ ping -c 2 192.168.10.130
PING 192.168.10.130 (192.168.10.130) 56(84) bytes of data:
64 bytes from 192.168.10.130: icmp_seq=1 ttl=64 time=0.030 ms
64 bytes from 192.168.10.130: icmp_seq=2 ttl=64 time=0.042 ms

--- 192.168.10.130 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1006ms
rtt min/avg/max/mdev = 0.030/0.036/0.042/0.006 ms
(kali@kali)~$
~$ sudo ip link delete core102lab0
(kali@kali)~$
~$ ip link show core102lab0 2>/dev/null || echo "Cleanup complete"
Cleanup complete
(kali@kali)~$
```

Report

I created a temporary dummy interface called core102lab0 after confirming no interface with that name already existed. The dummy type does not require a second adapter and does not alter the default route, making it safe on a single-VM setup. I brought the interface up and assigned two addresses from different /25 subnets of 192.168.10.0/24 — 192.168.10.10/25 and 192.168.10.130/25. ip -br addr show core102lab0 confirmed both addresses were attached, and ip route show dev core102lab0 showed the two directly-connected routes Linux created automatically, one per /25, each with the corresponding source address. Both ping commands returned 2/2 replies with 0% packet loss. The pings succeed without another computer because the kernel delivers traffic addressed to its own interface back up the local IP stack: the ICMP echo request is generated locally, matches a locally-configured address, and the stack answers it itself. This confirms the addresses and subnet masks are correctly configured and no routing fault exists. I then removed the interface with sudo ip link delete core102lab0, and the final verification printed "Cleanup complete" proving no residue remains.

Part 5 Variable Length Subnet Design

Allocate the largest requirement first. Use the smallest subnet that provides enough traditional usable addresses. The address block is 192.168.50.0/24.

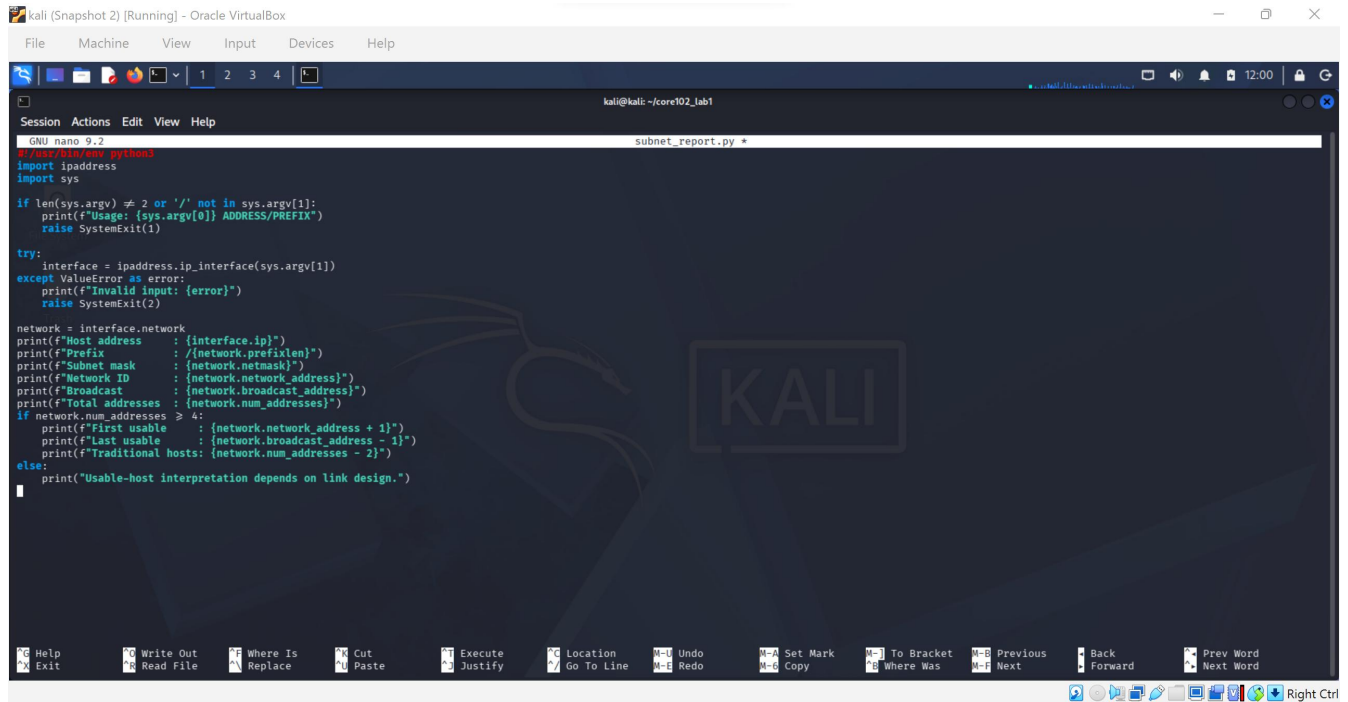
Group	Required hosts	Chosen prefix	Network ID	Usable range	Broadcast
Training hall	90	/25	192.168.50.0	1 – .126	192.168.50.127
Staff network	40	/26	192.168.50.128	129 – .190	192.168.50.191
Cyber range	20	/27	192.168.50.192	193 – .222	192.168.50.223
Management	10	/28	192.168.50.224	225 – .238	192.168.50.239
Router link	2	/30	192.168.50.240	241 – .242	192.168.50.243

Report

I designed the VLSM plan by ordering requirements from largest to smallest and choosing the smallest subnet that fits each. Training hall needs 90 hosts, so /25 (126 usable). Staff needs 40, so /26 (62). Cyber range needs 20, so /27 (30). Management needs 10, so /28 (14). The router-to-router link needs exactly 2, so /30 (2). Allocating sequentially from 192.168.50.0/24 without reusing ranges produced the layout above with no overlap. The remaining unallocated space is 192.168.50.244 – 192.168.50.255 — 12 addresses — enough for a /29 (6 usable) plus a /30 (2 usable), leaving 4 unused. Python's ipaddress module confirmed every network address, broadcast and usable-host count matched my manual plan.

Part 6 Subnet Verification Script

Evidence



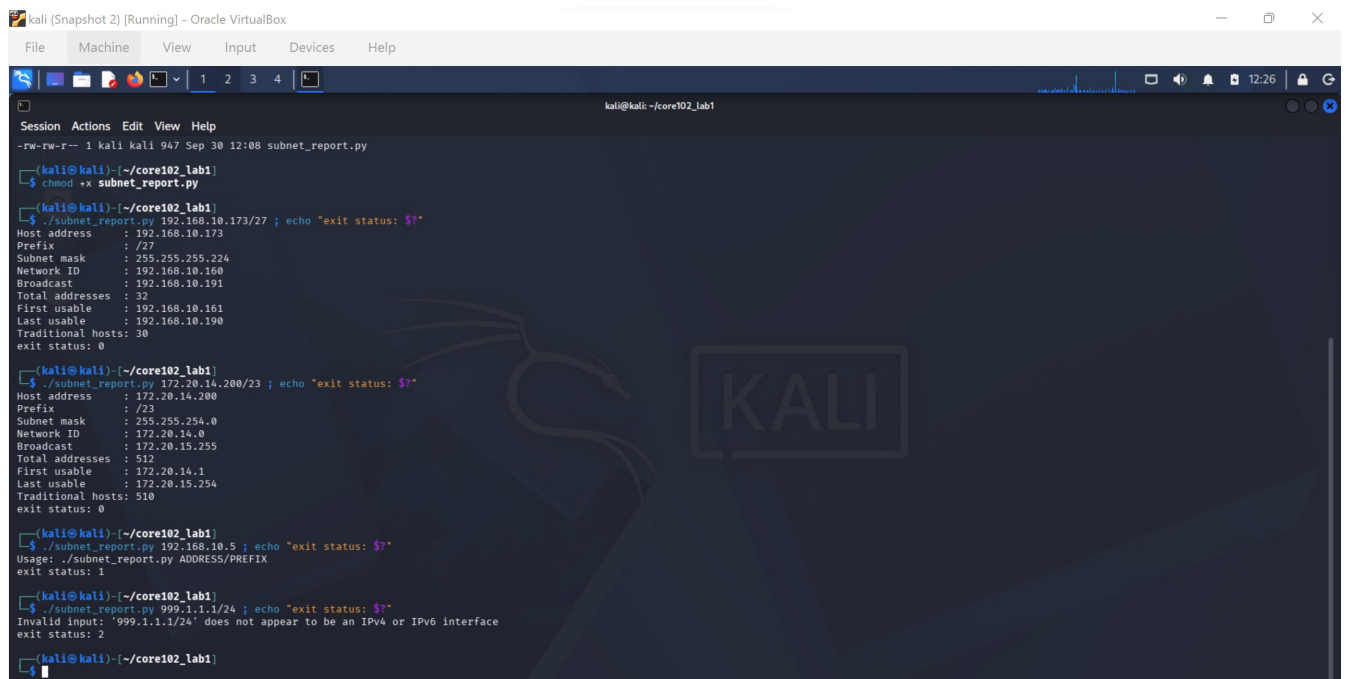
The screenshot shows a terminal window titled "kali (Snapshot 2) [Running] - Oracle VirtualBox". The terminal is running the nano text editor to create a file named "subnet_report.py". The script is written in Python and is designed to verify a subnet configuration. It takes two arguments: an IP address and a prefix. It checks if the arguments are provided, then uses the ipaddress module to determine the network details. The script prints out the host address, prefix, subnet mask, network ID, broadcast address, total addresses, first usable address, last usable address, and traditional hosts. It also includes a try-except block to handle invalid input.

```
GNU nano 9.2 subnet_report.py *
#!/usr/bin/env python3
import ipaddress
import sys

if len(sys.argv) != 2 or '/' not in sys.argv[1]:
    print(f"Usage: {sys.argv[0]} ADDRESS/PREFIX")
    raise SystemExit(1)

try:
    interface = ipaddress.ip_interface(sys.argv[1])
except ValueError as error:
    print(f"Invalid input: {error}")
    raise SystemExit(2)

network = interface.network
print(f"Host address      : {interface.ip}")
print(f"Prefix              : /{network.prefixlen}")
print(f"Subnet mask         : {network.netmask}")
print(f"Network ID          : {network.network_address}")
print(f"Broadcast           : {network.broadcast_address}")
print(f"Total addresses     : {network.num_addresses}")
if network.num_addresses > 4:
    print(f"First usable        : {network.network_address + 1}")
    print(f"Last usable         : {network.broadcast_address - 1}")
    print(f"Traditional hosts   : {network.num_addresses - 2}")
else:
    print("Usable-host interpretation depends on link design.")
```



The screenshot shows the same terminal window after the script has been executed several times. The terminal output shows the results of the script for different inputs. The first run shows the output for the IP address 192.168.10.173/27. The second run shows the output for the IP address 172.20.14.200/23. The third run shows the output for the IP address 192.168.10.5, which results in an "Invalid input" error because it does not contain a slash. The fourth run shows the output for the IP address 999.1.1.1/24, which results in an "Invalid input" error because it is not a valid IP address.

```
-rw-rw-r-- 1 kali kali 947 Sep 30 12:08 subnet_report.py

(kali@kali)-[~/core102_lab1]
$ chmod +x subnet_report.py

(kali@kali)-[~/core102_lab1]
$ ./subnet_report.py 192.168.10.173/27 ; echo "exit status: $?"
Host address      : 192.168.10.173
Prefix            : /27
Subnet mask       : 255.255.255.224
Network ID        : 192.168.10.160
Broadcast         : 192.168.10.191
Total addresses   : 32
First usable      : 192.168.10.161
Last usable       : 192.168.10.190
Traditional hosts : 30
exit status: 0

(kali@kali)-[~/core102_lab1]
$ ./subnet_report.py 172.20.14.200/23 ; echo "exit status: $?"
Host address      : 172.20.14.200
Prefix            : /23
Subnet mask       : 255.255.254.0
Network ID        : 172.20.14.0
Broadcast         : 172.20.15.255
Total addresses   : 512
First usable      : 172.20.14.1
Last usable       : 172.20.15.254
Traditional hosts : 510
exit status: 0

(kali@kali)-[~/core102_lab1]
$ ./subnet_report.py 192.168.10.5 ; echo "exit status: $?"
Usage: ./subnet_report.py ADDRESS/PREFIX
exit status: 1

(kali@kali)-[~/core102_lab1]
$ ./subnet_report.py 999.1.1.1/24 ; echo "exit status: $?"
Invalid input: '999.1.1.1/24' does not appear to be an IPv4 or IPv6 interface
exit status: 2

(kali@kali)-[~/core102_lab1]
$
```

Report

I wrote subnet report.py using Python's ip address module to accept a single ADDRESS/PREFIX argument and print the network properties used throughout this report. Two valid cases confirmed correct behaviour: 192.168.10.173/27 returned network ID 192.168.10.160, broadcast 192.168.10.191, 32 total addresses, first usable .161, last usable .190, 30 traditional hosts — exit status 0. 172.20.14.200/23 returned network 172.20.14.0, broadcast 172.20.15.255, 512 total addresses, 510 usable — exit status 0. Two invalid cases exercised the guard clauses. 192.168.10.5 was rejected because the argument contained no prefix, triggering the usage message and raise SystemExit (1). 999.1.1.1/24 was rejected inside the try block because ipaddress.ip_interface raised ValueError on the invalid octet, triggering the "Invalid input" message and raise SystemExit (2). Using two distinct exit codes lets callers distinguish between "user supplied the wrong argument format" (1) and "user supplied a syntactically valid argument with an invalid address value" (2), which is useful when the script is invoked from another program or CI pipeline.

Reflection answers:

1. Why can the historical class of an address differ from the prefix used in a modern network? Classful addressing dates from the early internet, when the first octet determined the default mask (A /8, B /16, C /24). Modern networks use CIDR, where any prefix can be assigned to any block. The historical class describes the mask the address would have had under old rules — it doesn't constrain today's prefix. For example, 10.24.6.9 is class A by first octet but can be deployed as /20.

2. Which mistake in subnet calculations is most likely to create an overlap?

Using the wrong block size when stepping between subnets. Dividing a /24 into /26 requires stepping by 64 (.0, .64, .128, .192) — stepping by 32 would overlap adjacent subnets. Always recompute $2^{\text{(host bits)}}$ and increment the network ID by that value.

3. How did command-line verification change or confirm your manual calculations?

Python's `ipaddress` confirmed every network ID, broadcast and usable-host count I calculated by hand, including the /27 subnet containing 192.168.10.173 and the VLSM allocations. It also caught edge cases: the script correctly rejects a bare address with no prefix, which is easy to miss manually. Using both methods gave two independent checks on the same result.

ZIP: contains only `subnet_report.py`